

gmpfrxx_mkII Reference Manual

A header-only C++17/20 wrapper for GMP, MPFR, and MPC

Maho Nakata

RIKEN Cluster for Pioneering Research

Chiba Institute of Technology

Version 0.0.1

Abstract

gmpfrxx_mkII is a from-scratch, header-only C++17/20 wrapper for GMP, MPFR, and MPC. It does not depend on `gmpxx.h` or `libgmpxx`, and does not bridge to any pre-existing C++ wrapper. This manual documents the public API surface as exposed by the project's detail headers. Internal evaluation machinery (expression-node evaluators, allocator-aware materialization paths) is mentioned only where its observable behavior matters to the caller.

Contents

1	Overview	3
1.1	Design properties	3
1.2	Cross-family arithmetic policy	3
2	Build and Installation	3
2.1	Configuration	4
2.2	Source distribution target	4
2.3	Compile-time options	4
2.4	Version information	4
3	Headers and Link Targets	4
3.1	Header rules	5
3.2	Linkage examples	5
4	Construction and Precision	5
4.1	Default construction	5
4.2	Numeric-value constructors	6
4.3	The <code>with_precision</code> factories	7
4.4	Value setters and accessors	8
4.5	String construction	10
4.6	Increment, decrement, and <code>swap</code>	10
5	Defaults and Environment Variables	10
5.1	GMP MPF defaults	10
5.2	MPFR defaults	11
5.3	MPC defaults	12

6	Expression Templates	12
6.1	Precision policy	12
6.2	Allocation behavior	13
6.3	Unary simplification	13
6.4	Direct-routed compound assignments	13
7	Mixed-Type Promotion	13
7.1	GMP family	13
7.2	MPFR/MPC family	13
7.3	Forbidden combinations	14
8	Scalar Operands	14
8.1	Accepted scalars	14
8.2	Rejected scalars	14
8.3	Internal normalization	14
8.4	The scalar/scalar boundary	14
8.5	Standard-library interoperability	14
9	Compound Assignment	15
9.1	Floating-point family	15
9.2	Integer-specific operators	15
9.3	Free functions for <code>mpz_class</code>	16
10	User-Defined Literals	17
11	String and Stream I/O	17
11.1	Exact types (<code>mpz_class</code> , <code>mpq_class</code>)	17
11.2	MPF	18
11.3	MPFR	18
11.4	MPFC complex	18
11.5	Stream extraction (<code>operator»</code>)	18
12	Math Functions	19
12.1	MPF (<code>gmpxx::mpf_class</code>)	19
12.2	MPFC (<code>gmpxx::mpfc_class</code>)	20
12.3	MPFR (<code>mpfrxx::mpfr_class</code>)	20
12.4	MPC (<code>mpfrxx::mpc_class</code>)	22
13	Random Number API	22
13.1	GMP random class (<code>gmpxx::gmp_randclass</code>)	23
13.2	MPFR random class (<code>mpfrxx::gmp_randclass</code>)	23
14	Examples	24
15	Limitations and Notes	25
A	Public Symbol Index	26

1 Overview

`gmpfrxx_mkII` exposes the following arbitrary-precision numeric types:

- `gmpxx::mpz_class` — exact integer (GMP `mpz_t`).
- `gmpxx::mpq_class` — exact rational (GMP `mpq_t`).
- `gmpxx::mpf_class` — GMP MPF floating point.
- `gmpxx::mpfc_class` — GMP-only complex, backed by two `mpf_class` components. Independent of MPC.
- `mpfrxx::mpfr_class` — MPFR correctly-rounded floating point.
- `mpfrxx::mpc_class` — MPC correctly-rounded complex.
- `mpfrxx::mpz_class`, `mpfrxx::mpq_class` — aliases for the corresponding `gmpxx` exact types (same type, not subclasses).

1.1 Design properties

- Header-only. All public APIs live under `include/`.
- Two public namespaces: `gmpxx` (GMP family) and `mpfrxx` (MPFR/MPC family). Internal facilities live in `gmpfrxx_mkII::detail` and are not part of the public API.
- Strict header separation. Code that needs only GMP can include `<gmpxx_mkII.h>` alone and never sees MPFR or MPC headers or symbols.
- Arithmetic operators (`+`, `-`, `*`, `/`, unary `+`, unary `-`) return expression nodes, not eager values. Materialization happens at construction or assignment.
- Allocation-aware evaluation: a left-associated addition chain such as `dst = a + b + c + d` performs zero allocations; a two-sided expression such as `dst = (a + b) * (c + d)` uses exactly one temporary.
- Aliasing-safe assignment: when the destination appears in the source expression, evaluation falls back to a temporary path.
- No eager arithmetic operator returning the underlying numeric type is defined; the only public surface is the expression-template surface.

1.2 Cross-family arithmetic policy

- `mpz`, `mpq`, and `mpf` freely promote within the GMP family.
- `mpz`, `mpq`, `mpfr`, and `mpc` freely promote within the MPFR/MPC family. `mpfr` promotes to `mpc` when at least one operand is `mpc`.
- `mpf` *must not* be combined with `mpfr` or `mpc`; this is enforced at compile time. `mpfc_class` (GMP-only complex) must not be combined with `mpfr_class` or `mpc_class` either.

2 Build and Installation

The build system is CMake. The project requires GMP, MPFR, and MPC at configure time for the full repository; MPFR-only and GMP-only consumers exist at the include and link-target boundary (see Section 3).

2.1 Configuration

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
```

CMake finder modules `cmake/FindGMP.cmake`, `cmake/FindMPFR.cmake`, and `cmake/FindMPC.cmake` are bundled.

2.2 Source distribution target

A `dist` target produces a clean source tarball using `git archive HEAD`:

```
cmake --build build --target dist
# produces build/gmpfrxx_mkII-0.0.1.tar.xz
```

The archive top directory is `gmpfrxx_mkII-0.0.1/`. The target is available only when both Git and `xz` are detected by CMake. Because the implementation is `git archive HEAD`, uncommitted changes are not included until committed.

2.3 Compile-time options

GMPFRXX_MKII_ASSUME_FIXED_PRECISION_FASTPATH

Define this preprocessor macro before including any wrapper header (or via `-D`) to enable a fast-path that assumes all `mpf_class`, `mpfr_class`, and `mpc_class` operands in expressions share the same precision. The macro flips `gmpfrxx_mkII::detail::build_options::assume_fixed_precision_fastpath` (a `constexpr bool` flag) to `true`, which feeds into evaluation decisions. Default: `off`.

2.4 Version information

The build system generates a private header `detail/version.hpp` from `detail/version.hpp.in`, embedding the Git commit hash. The following accessors are provided in both `gmpxx` and `mpfrxx` namespaces:

```
namespace gmpxx /* and namespace mpfrxx */ {
    constexpr const char* git_commit_hash() noexcept;
    std::ostream& print_git_commit_hash(std::ostream& out);
}
```

When the version header is unavailable (e.g. in a sandboxed build), `git_commit_hash()` returns the literal `"unknown"`.

3 Headers and Link Targets

The library exposes three concentric public headers and four CMake `INTERFACE` targets. Choose the smallest pair sufficient for your translation unit.

Header	CMake target	Links
<code><gmpxx_mkII.h></code>	<code>gmpxx_mkII</code>	GMP
<code><mpfrxx_mkII.h></code>	<code>mpfrxx_mkII</code>	GMP, MPFR
<code><mpcxx_mkII.h></code>	<code>mpcxx_mkII</code>	MPC (requires MPFR header first)
<code><gmpfrxx_mkII.h></code>	<code>gmpfrxx_mkII</code>	GMP, MPFR, MPC (aggregator)

3.1 Header rules

- `<gmpxx_mkII.h>` is GMP-only. It must not include or expose any MPFR or MPC header, type, or symbol. Compile-fail tests enforce this.
- `<mpfrxx_mkII.h>` provides MPFR (`mpfr_class`) but does not expose `mpc_class` or `mpfc_class`.
- `<mpcxx_mkII.h>` provides `mpc_class` and *requires* that `<mpfrxx_mkII.h>` has been included beforehand. It does not include MPFR implicitly. A static assertion enforces include order.
- `<gmpfrxx_mkII.h>` is the convenience aggregator. It pulls in all of the above, including `<mpcxx_mkII.h>`.

3.2 Linkage examples

```
# GMP-only consumer
target_link_libraries(myapp PRIVATE gmpxx_mkII)

# MPFR consumer (no MPC)
target_link_libraries(myapp PRIVATE mpfrxx_mkII)

# MPC consumer (must include both MPFR and MPC headers)
target_link_libraries(myapp PRIVATE mpcxx_mkII)

# Everything
target_link_libraries(myapp PRIVATE gmpfrxx_mkII)
```

4 Construction and Precision

Numeric precision is selected through one of three mechanisms:

- Implicit, using the family’s process-global wrapper default (Section 5).
- Explicit, by passing `mp_bitcnt_t` or `mpfr_prec_t` as the second constructor argument.
- Through the `with_precision(...)` static factories on `mpf_class`, `mpfc_class`, and `mpc_class` (and presumably `mpfr_class`).

All numeric types reject `bool` construction (= `delete`). `mpz_class`, `mpq_class`, and `mpf_class` additionally accept `__int128_t`, `__uint128_t` where the GMP API allows exact conversion. Note that `__int128` as an *expression-template scalar leaf* is rejected at compile time; the two paths are distinct.

4.1 Default construction

Default construction yields a zero-valued object at the current wrapper-owned default precision.

```
gmpxx::mpz_class z; // value 0 (precision is irrelevant for mpz)
gmpxx::mpq_class q; // value 0/1
gmpxx::mpf_class f; // 0.0 at default MPF precision
gmpxx::mpfc_class fc; // (0,0) at default MPF precision per component
mpfrxx::mpfr_class r; // 0.0 at default MPFR precision
mpfrxx::mpc_class c; // (0,0) at default MPC precision per component
```

4.2 Numeric-value constructors

mpz_class

```
gmpxx::mpz_class a(42); // any non-bool integral type
gmpxx::mpz_class b(__int128_t{...}); // 128-bit signed
gmpxx::mpz_class c(__uint128_t{...}); // 128-bit unsigned
gmpxx::mpz_class d(double{3.0}); // explicit; truncates fractional part
gmpxx::mpz_class e("ff", 16); // string + base, base default 10
gmpxx::mpz_class f(std::string("123"));
gmpxx::mpz_class g(other_mpz); // copy
gmpxx::mpz_class h(some_mpz_srcptr); // explicit, from raw mpz_t
explicit mpz_class(const mpf_class&); // truncates
explicit mpz_class(const mpq_class&); // truncates
```

mpq_class

```
gmpxx::mpq_class a(42); // integral
gmpxx::mpq_class b(some_mpz); // integer numerator
gmpxx::mpq_class c(num_mpz, den_mpz); // num/den, canonicalized
gmpxx::mpq_class d(355, 113); // integral pair
gmpxx::mpq_class e(double{0.5}); // explicit
gmpxx::mpq_class f("355/113"); // string, canonicalized
explicit mpq_class(const mpf_class&); // exact MPF -> rational
```

mpf_class

mpf_class provides numeric-value constructors at the default precision and overloads taking an explicit precision in mp_bitcnt_t.

```
gmpxx::mpf_class a(1.5); // double, default precision
gmpxx::mpf_class b(1.5, 256); // double, 256 bits
gmpxx::mpf_class c(int{42}); // any non-bool integral
gmpxx::mpf_class d(42L, 256); // integral + precision
gmpxx::mpf_class e(some_mpz); // exact int -> mpf
gmpxx::mpf_class f(some_mpq, 256); // exact rational -> mpf at 256
gmpxx::mpf_class g(other_mpf, 256); // requantize precision
gmpxx::mpf_class h("3.14159"); // string at default precision
gmpxx::mpf_class i("3.14159", 256); // string + precision
gmpxx::mpf_class j("3.14159", 256, 10); // string + precision + base
```

Constructor diagnostics. Invalid string input raises `std::invalid_argument`.

mpfc_class (GMP-only complex)

```
gmpxx::mpfc_class c(real_mpf, imag_mpf); // pair of mpf_class
explicit mpfc_class(const mpf_class&); // real-only
explicit mpfc_class(double{1.5}); // real-only scalar
gmpxx::mpfc_class d(1.5, -1.5); // scalar pair
gmpxx::mpfc_class e =
    gmpxx::mpfc_class::with_precision(256, 1.0, -1.0);
```

mpfr_class (MPFR real)

```
mpfrxx::mpfr_class a(1.5); // double, default precision
mpfrxx::mpfr_class b(1.5, 256); // double, 256 bits
mpfrxx::mpfr_class c(int{42}); // any non-bool integral
mpfrxx::mpfr_class d(42L, 256); // integral + precision
mpfrxx::mpfr_class e(some_mpz); // mpz_class, default prec
mpfrxx::mpfr_class f(some_mpz, 256); // mpz_class + precision
mpfrxx::mpfr_class g(some_mpq, 256); // mpq_class + precision
mpfrxx::mpfr_class h(other_mpfr, 256); // requantize precision
mpfrxx::mpfr_class i("3.14159"); // string at default prec
mpfrxx::mpfr_class j("3.14159", 256); // string + precision
mpfrxx::mpfr_class k("3.14159", 256, 10); // string + precision + base
mpfrxx::mpfr_class l(some_mpfr_srcptr); // explicit, raw mpfr_t
mpfrxx::mpfr_class m(some_mpfr_srcptr, 256); // raw mpfr_t + precision
```

Constructor diagnostics. String constructors throw `std::invalid_argument` on parse failure. `bool` construction and `bool` assignment are = `deleted`. Unlike `mpz_class`, `mpfr_class` does *not* accept `__int128_t` or `__uint128_t` as a constructor argument.

mpc_class (MPC complex)

`mpc_class` has *no* scalar-value constructor. Use `with_precision` to set the value through doubles, or pass an `mpfr_class` pair.

```
mpfrxx::mpc_class a(real_mpfr, imag_mpfr); // pair of mpfr_class
auto b = mpfrxx::mpc_class::with_precision(256);
auto c = mpfrxx::mpc_class::with_precision(256, 256); // (re,im) prec
auto d = mpfrxx::mpc_class::with_precision(256, /*re=*/1.0, /*im=*/2.0);
auto e = mpfrxx::mpc_class::with_precision(256, 128, 1.0, 2.0); // mixed prec
```

4.3 The with_precision factories

```
// MPF
auto f0 = gmpxx::mpf_class::with_precision(256);
auto f1 = gmpxx::mpf_class::with_precision(256, /*value=*/1.5);

// MPFR (only with_precision(prec) and with_precision(prec, double))
auto r0 = mpfrxx::mpfr_class::with_precision(256);
auto r1 = mpfrxx::mpfr_class::with_precision(256, /*value=*/1.5);

// MPFC: per-component or shared precision
auto c0 = gmpxx::mpfc_class::with_precision(256);
auto c1 = gmpxx::mpfc_class::with_precision(/*re=*/256, /*im=*/128);
auto c2 = gmpxx::mpfc_class::with_precision(256, /*re=*/1.0, /*im=*/-1.0);

// MPC: per-component or shared precision
auto z0 = mpfrxx::mpc_class::with_precision(256);
auto z1 = mpfrxx::mpc_class::with_precision(/*re=*/256, /*im=*/128);
auto z2 = mpfrxx::mpc_class::with_precision(256, /*re=*/1.0, /*im=*/2.0);
```

4.4 Value setters and accessors

Common accessors

Every numeric class exposes raw GMP/MPFR/MPC pointer access for interop:

```
const mpz_t& mpz_class::mpz_data() const; mpz_t& mpz_data();
const mpq_t& mpq_class::mpq_data() const; mpq_t& mpq_data();
const mpf_t& mpf_class::mpf_data() const; mpf_t& mpf_data();
const mpc_t& mpc_class::mpc_data() const; mpc_t& mpc_data();
// mpfr_class follows the same pattern with mpfr_data().
```

`get_mpz_t()` / `get_mpq_t()` / `get_mpf_t()` spellings are also provided for compatibility with the `gmpxx` idiom.

`mpz_class` / `mpq_class` accessors and setters

```
mpz_class a;
a.set("42"); a.set(std::string("42")); // throws on failure
int rc = a.set_str("42", 10); // non-throwing; rc==0 on success
std::string s = a.get_str(/*base=*/10);
std::string t = a.to_string(10);
double x = a.get_d();
unsigned long u = a.get_ui();
signed long v = a.get_si();
bool ok = a.fits_slong_p(); // also fits_sint_p, fits_sshort_p,
                           // fits_uint_p, fits_ulong_p, fits_ushort_p

mpq_class q;
q.canonicalize();
mpz_class num = q.get_num();
mpz_class den = q.get_den();
mpz_srcptr np = q.get_num_mpz_t();
mpz_srcptr dp = q.get_den_mpz_t();
double d = q.get_d();
```

`mpf_class` accessors and setters

```
mpf_class f = mpf_class::with_precision(256);
f.set(2.5);
f.set("3.14159"); // throws on failure
int rc = f.set_str("3.14159", 10); // non-throwing

double d = f.to_double(); // also get_d()
unsigned long u = f.get_ui();
signed long v = f.get_si();
mp_bitcnt_t p = f.precision(); // also get_prec()

f.set_prec(512); // change effective precision (preserves value)
f.set_prec_raw(512); // raw precision change (no value preservation)
f.div_2exp(8); // f /= 2^8
f.mul_2exp(8); // f *= 2^8
f.set_epsilon(); // f := 2^-(prec-1)

bool ok = f.fits_slong_p(); // fits_sint_p, fits_sshort_p,
                           // fits_uint_p, fits_ulong_p, fits_ushort_p
```



```

std::string s = f.to_string(/*n_digits=*/0);
std::string g = f.get_str();
mp_exp_t exp;
std::string g2 = f.get_str(exp, /*base=*/10, /*n_digits=*/0);

```

mpfr_class accessors and setters

```

mpfr_class r = mpfr_class::with_precision(256);
r.set(2.5);
r.set(int{42}); // any non-bool integral
r.set("3.14159"); // throws on failure
int rc = r.set_str("3.14159", 10); // non-throwing
int rc2 = r.set_str("0xff.0p-4", 0); // base 0 = auto-detect

double d = r.to_double(); // also get_d()
unsigned long u = r.get_ui();
signed long v = r.get_si();
mpfr_prec_t p = r.precision(); // also get_prec()

r.set_prec(512); // round-aware precision change

bool ok = r.fits_slong_p(); // fits_sint_p, fits_sshort_p,
                           // fits_uint_p, fits_ulong_p, fits_ushort_p

// Explicit conversion to bool (true iff non-zero), mpz_class, mpq_class.
bool nz = static_cast<bool>(r);
mpz_class as_z(static_cast<mpz_class>(r)); // truncates
mpq_class as_q(static_cast<mpq_class>(r)); // canonicalized

std::string s = r.to_string(/*n_digits=*/0);
std::string g = r.get_str();
mpfr_exp_t e;
std::string g2 = r.get_str(e, /*base=*/10, /*n_digits=*/0);

// Static accessors mirroring the namespace-level defaults.
mpfr_prec_t dp = mpfr_class::default_precision(); // == default_precision_bits()
mpfr_rnd_t dr = mpfr_class::default_rounding(); // == default_rounding_mode()

```

mpfr_class has only set_prec() (round-aware, via mpfr_prec_round); there is no set_prec_raw on this class.

mpfc_class component access

```

gmpxx::mpfc_class c(1.0, 2.0);
mpf_class& re = c.real(); // mutable reference
const mpf_class& ci = c.imag(); // const reference
c.real(other_mpf); // setter
c.imag(other_mpf);
mp_bitcnt_t pr = c.real_precision();
mp_bitcnt_t pi = c.imag_precision();
mp_bitcnt_t pp = c.precision(); // == max(pr, pi)

```

mpc_class component access

```
mpfrxx::mpc_class z = mpfrxx::mpc_class::with_precision(256, 1.0, 2.0);
double re = z.real_to_double();
double im = z.imag_to_double();
mpfr_prec_t pr = z.real_precision();
mpfr_prec_t pi = z.imag_precision();
mpfr_prec_t pp = z.precision(); // common precision if equal,
                                // else min(pr, pi)
mpc_rnd_t rnd = mpfrxx::mpc_class::default_rounding();
```

For component reads on `mpc_class` use the math-side helpers `mpfrxx::real(z)`, `mpfrxx::imag(z)`; both return `mpfr_class` (Section 12.4).

4.5 String construction

`mpz_class`, `mpq_class`, and `mpf_class` all accept `const char*` and `std::string`. Successful `mpq_class` parses are canonicalized; zero denominators are rejected.

```
gmpxx::mpz_class z("ff", 16); // 255
gmpxx::mpq_class q("355/113"); // canonicalized
gmpxx::mpf_class f("3.14159", 256); // explicit precision
mpfrxx::mpfr_class r("1.0e-100", 1024); // explicit precision
```

Throwing vs. non-throwing. Constructors and assignment operators that take a string (`const char*` or `std::string`) throw `std::invalid_argument` on parse failure. The `set_str()` member returns an `int` (0 on success, non-zero on failure) and preserves the destination value when parsing fails. The `set()` member throws on failure.

4.6 Increment, decrement, and swap

`mpz_class`, `mpq_class`, and `mpf_class` support prefix and postfix `operator++` / `operator--`. All wrapper classes provide a non-throwing member `swap()` and a free `swap()` that delegates to it.

5 Defaults and Environment Variables

Each numeric family has a process-global, wrapper-owned default. Defaults are loaded from environment variables at static-initialization time and can be reloaded or overwritten at run-time. The wrapper does *not* call `mpf_set_default_prec()`; GMP's process-global default is left untouched.

5.1 GMP MPF defaults

Header: `<gmpxx_mkII.h>`

Initial default precision: **512 bits**.

```
namespace gmpxx {
    mp_bitcnt_t default_mpf_precision_bits();
    void set_default_mpf_precision_bits(mp_bitcnt_t bits);
    void reload_default_mpf_precision_bits_from_environment();
}
```

Environment variable: `MPFXX_DEFAULT_PREC_BITS`.

Note on `mpf_get_prec()`

GMP rounds requested MPF precision up to its internal limb allocation, so `mpf_get_prec()` may return a value greater than the requested default. The wrapper preserves the *requested* default for new constructions; existing destinations preserve their effective GMP precision on assignment.

5.2 MPFR defaults

Header: `<mpfrxx_mkII.h>`

Initial default precision: **512 bits**.

Initial default rounding mode: `MPFR_RNDN`.

```
namespace mpfrxx {
    // Aggregate accessor.
    struct mpfr_default_options {
        mpfr_prec_t precision_bits;
        mpfr_exp_t emin;
        mpfr_exp_t emax;
        mpfr_rnd_t rounding_mode;
    };
    mpfr_default_options default_options();

    // Precision.
    mpfr_prec_t default_precision_bits();
    mpfr_prec_t default_prec(); // alias for default_precision_bits
    void set_default_precision_bits(mpfr_prec_t);

    // Rounding mode.
    mpfr_rnd_t default_rounding_mode();
    void set_default_rounding_mode(mpfr_rnd_t);

    // Exponent range.
    mpfr_exp_t default_emin();
    mpfr_exp_t default_emax();
    void set_default_exponent_range(mpfr_exp_t emin, mpfr_exp_t emax);

    // Reload from environment.
    void reload_mpfr_defaults_from_environment();
}
```

Environment variables (parsed at startup or on reload):

Variable	Effect
<code>MPFRXX_DEFAULT_PRECISION_BITS</code>	Wrapper default precision in bits
<code>MPFRXX_EMIN</code>	Default exponent minimum
<code>MPFRXX_EMAX</code>	Default exponent maximum
<code>MPFRXX_ROUNDING_MODE</code>	One of: <code>RNDN</code> , <code>RNDZ</code> , <code>RNDU</code> , <code>RNDD</code> , <code>RNDA</code> , <code>RNDF</code> , or the <code>MPFR_*</code> spellings

Invalid values fall back to the documented defaults; explicit setters silently no-op when given out-of-range arguments (precision below `MPFR_PREC_MIN`, or `emin > emax`). The exponent-range pair is loaded atomically: if only one of `MPFRXX_EMIN` or `MPFRXX_EMAX` is set, it is applied only if the implied range is self-consistent. Wrapper expression evaluation guards the MPFR process-global exponent range with the wrapper-owned defaults via an RAI guard and restores the previous values afterward.

5.3 MPC defaults

Header: `<mpfrxx_mkII.h>` (the `mpc_class` type itself lives in `<mpcxx_mkII.h>`).

MPC defaults inherit from MPFR defaults unless `MPFRXX_MPC_*` variables override them. Real and imaginary parts may be configured independently for both precision and rounding.

```
namespace mpfrxx {
    options_t default_mpc_options();

    mpfr_prec_t default_mpc_precision_bits();
    mpfr_prec_t default_mpc_real_precision_bits();
    mpfr_prec_t default_mpc_imag_precision_bits();
    void set_default_mpc_precision_bits(mpfr_prec_t bits);
    void set_default_mpc_precision_bits(mpfr_prec_t real_bits,
                                         mpfr_prec_t imag_bits);

    mpc_rnd_t default_mpc_rounding_mode();
    mpfr_rnd_t default_mpc_real_rounding_mode();
    mpfr_rnd_t default_mpc_imag_rounding_mode();
    void set_default_mpc_rounding_mode(mpc_rnd_t rounding);
    void set_default_mpc_rounding_mode(mpfr_rnd_t real_rnd,
                                         mpfr_rnd_t imag_rnd);

    void reload_mpc_defaults_from_environment();
}
```

Environment variables:

MPFRXX_MPC_DEFAULT_PRECISION_BITS	Common default precision
MPFRXX_MPC_REAL_PRECISION_BITS	Real-part precision
MPFRXX_MPC_IMAG_PRECISION_BITS	Imag-part precision
MPFRXX_MPC_ROUNDING_MODE	Combined rounding mode
MPFRXX_MPC_REAL_ROUNDING_MODE	Real-part rounding
MPFRXX_MPC_IMAG_ROUNDING_MODE	Imag-part rounding

6 Expression Templates

All public arithmetic operators (+, -, *, /, unary +, unary -) return expression nodes from `gmpfrxx_mkII::detail` rather than eager numeric values. Materialization occurs in two contexts:

1. Construction of a numeric class from the expression.
2. Assignment of the expression to an existing numeric object.

6.1 Precision policy

- **New materialization** (constructing a fresh object from an expression) uses the maximum precision among the relevant object leaves in the expression.
- **Assignment to an existing object** preserves the destination's precision; the expression is evaluated at the destination's precision.
- Exact `mpz/mpq` leaves contribute *zero* precision to mixed exact/MPF or exact/MPFR materialization. The MPF or MPFR leaf precision drives the result.

- For `mpc_class`, real and imaginary precisions are tracked separately. New materialization uses the per-component max across leaves; assignment preserves per-component destination precision.

6.2 Allocation behavior

- Simple leaf-only assignments (`dst = a + b`) allocate zero times.
- Left-associated addition chains (`dst = a + b + c + d`) allocate zero times.
- Two-sided compound expressions (`dst = (a + b) * (c + d)`) use exactly one temporary `mpfr_t` (or `mpf_t`, etc.).
- Aliasing assignments (where the destination appears in the source) fall back to the temporary path; safety is preserved.

6.3 Unary simplification

A double negation `-(-expr)` is simplified to a positive-tag node `pos_op(expr)` during expression construction. `mpz_class` additionally supports bitwise complement `operator~`.

6.4 Direct-routed compound assignments

The `mpz_class` compound assignments `+=` and `-=` for expressions of shape `mpz * mpz` (or with a small unsigned scalar) are routed directly to GMP's `mpz_addmul`, `mpz_submul`, `mpz_addmul_ui`, or `mpz_submul_ui`. The generic compound overload is excluded from these shapes to ensure the fast path is used.

7 Mixed-Type Promotion

7.1 GMP family

- `mpz + mpz → mpz_class`.
- `mpq + mpq → mpq_class`; `mpz + mpq`, division in either direction `→ mpq_class`.
- `mpz` or `mpq` mixed with `mpf` `→ mpf_class`. Exact `mpz` leaves go through `mpf_set_z()`. Exact `mpq` leaves are evaluated by computing numerator and denominator as MPF values and dividing (GMP MPF has no `mpf_set_q()`).
- `mpz` or `mpq` mixed with `mpfc` `→ mpfc_class`.

7.2 MPFR/MPC family

- `mpz` or `mpq` mixed with `mpfr` `→ mpfr_class`. `mpz` leaves go through `mpfr_set_z()`; `mpq` leaves go through `mpfr_set_q()`.
- Exact `mpz`- or `mpq`-result subexpressions embedded in an MPFR expression are evaluated in the exact GMP evaluator first, then converted once with `mpfr_set_z()` or `mpfr_set_q()`. This avoids precision loss in intermediate exact arithmetic.
- `mpc` promotes any operand of type `mpfr`, `mpz`, or `mpq` when at least one operand is `mpc`. Conversions go through `mpc_set_fr()`, `mpc_set_z()`, and `mpc_set_q()` respectively.

7.3 Forbidden combinations

The following are rejected at compile time:

- `mpf_class` combined with `mpfr_class` in either order.
- `mpf_class` combined with `mpc_class` in either order.
- `mpfc_class` combined with `mpfr_class` or `mpc_class`.

8 Scalar Operands

Standard built-in scalars participate in expression templates, subject to strict acceptance rules.

8.1 Accepted scalars

- Signed and unsigned standard integral types, *except* `bool`.
- `float` and `double`.

8.2 Rejected scalars

- `bool` (compile-fail test enforced).
- `long double` (compile-fail test enforced).
- `__int128` and `unsigned __int128` (compile-fail test).

8.3 Internal normalization

Accepted scalars are normalized at the expression-leaf level to one of three ABI-stable types:

Source kind	Normalized leaf type
signed integral	<code>std::int64_t</code>
unsigned integral	<code>std::uint64_t</code>
float, double	<code>double</code>

`std::uint64_t` max, `std::int64_t` min, and `std::int64_t` max round-trip exactly through the integer conversion path; the signed minimum is not negated during conversion.

8.4 The scalar/scalar boundary

Expression-template operators always require at least one wrapper object or wrapper expression operand. Two raw scalars never form an expression node; this prevents the detail operators from accidentally applying to ordinary scalar arithmetic.

8.5 Standard-library interoperability

`std::common_type`

Explicit `std::common_type` specializations are provided for `mpz_class`, `mpq_class`, `mpf_class`, and `mpfc_class` paired with each of the following builtin scalar types, both directions: `char`, `signed char`, `unsigned char`, `wchar_t`, `char16_t`, `char32_t`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float`, and `double`. The result type in each case is the wrapper class. Cross-class specializations (`mpz/mpq`, `mpz/mpf`, `mpq/mpf`, and the corresponding `mpfc` pairs) follow the promotion table in Section 7.

`std::common_type` is also specialized for the expression-template node types declared in `gmpfrxx_mkII::detail` so that mixing `object_leaf`, `scalar_leaf`, `unary_expr`, and `binary_expr` resolves to the underlying `result_type`.

`mpfrxx::mpfr_class` has `std::common_type` specializations for the three pairings (`mpfr_class`, `mpz_class`), (`mpfr_class`, `mpq_class`), and (`mpfr_class`, `mpfr_class`), all yielding `mpfr_class`. There are no `std::common_type` specializations between `mpfr_class` and builtin scalar types, nor for `mpc_class`.

`std::numeric_limits`

The following minimal specializations are provided:

Type	is_specialized	is_signed	is_integer	is_exact	is_bounded
<code>gmpxx::mpz_class</code>	true	true	true	true	false
<code>gmpxx::mpq_class</code>	true	true	false	true	false
<code>gmpxx::mpf_class</code>	true	true	false	false	false

`is_modulo` is false for all three. No `numeric_limits` specialization is provided for `mpfc_class`, `mpfr_class`, or `mpc_class`; for those types `numeric_limits<T>::is_specialized` is false.

9 Compound Assignment

9.1 Floating-point family

`mpf_class`, `mpfc_class`, `mpfr_class`, and `mpc_class` provide `operator+=`, `operator-=`, `operator*=`, and `operator/=` accepting the same operand families as the binary operators (object, expression, supported scalar). Compound assignment *always* preserves the destination's precision regardless of operand precision.

```
mpfrxx::mpfr_class x = mpfrxx::mpfr_class::with_precision(256, 1.0);
mpfrxx::mpfr_class y = mpfrxx::mpfr_class::with_precision(1024, 2.0);
x += y; // x stays at 256 bits, evaluated at 256 bits
x *= 3; // scalar
x -= (y * y + 1); // expression RHS
```

`mpf_class` and `mpfr_class` additionally support power-of-two scaling via `operator<=` and `operator>=` (shift by unsigned long), which delegate to `mpf_mul_2exp/mpf_div_2exp` or `mpfr_mul_2ui/mpfr_div_2ui` respectively. Pre- and post-increment / decrement (`++`, `-`) are also defined for `mpfr_class` in both lvalue and rvalue forms.

9.2 Integer-specific operators

`mpz_class` exposes the full GMP integer-arithmetic surface as public operators.

Arithmetic compound assignment

`mpz_class` supports `+=`, `-=`, `*=`, `/=` with operand types `mpz_class`, supported integral and floating-point scalars, and expression nodes. The `mpz_class += mpz * mpz` and `mpz_class -= mpz * mpz` shapes (and scalar-multiplied variants) are routed directly to GMP's `mpz_addmul` / `mpz_submul` (or `mpz_addmul_ui` / `mpz_submul_ui`) fast paths; the generic compound overload is specifically excluded from these shapes to ensure dispatch.

Modulo

```
mpz_class operator%(const mpz_class&, const mpz_class&);
mpz_class operator%(const mpz_class&, double);
mpz_class operator%(double, const mpz_class&);
template <Integral I> mpz_class operator%(const mpz_class&, I);
template <Integral I> mpz_class operator%(I, const mpz_class&);

mpz_class& operator%=(mpz_class&, const mpz_class&);
mpz_class& operator%=(mpz_class&, double);
template <Integral I> mpz_class& operator%=(mpz_class&, I);
```

Bitwise

```
mpz_class operator~(const mpz_class&); // bitwise complement
mpz_class& operator&=(mpz_class&, /* mpz, scalar, or expression */);
mpz_class& operator|=(mpz_class&, /* mpz, scalar, or expression */);
mpz_class& operator^=(mpz_class&, /* mpz, scalar, or expression */);
```

Shift

```
mpz_class& operator<<=(mpz_class&, unsigned long);
mpz_class& operator>>=(mpz_class&, unsigned long);
```

Increment / decrement

Pre- and post-increment / decrement are defined for `mpz_class`, `mpq_class`, and `mpf_class`, in both lvalue and rvalue forms.

9.3 Free functions for `mpz_class`

```
namespace gmpxx {
    mpz_class abs(const mpz_class&);
    mpz_class abs(/* expression node */); // SFINAE-restricted

    mpz_class sqrt(const mpz_class&); // floor of integer sqrt

    mpz_class gcd(const mpz_class&, const mpz_class&);
    mpz_class gcd(/* mixed lhs */, /* mixed rhs */);

    mpz_class lcm(const mpz_class&, const mpz_class&);
    mpz_class lcm(/* mixed lhs */, /* mixed rhs */);

    mpz_class factorial (const mpz_class&);
    mpz_class primorial (const mpz_class&);
    mpz_class fibonacci (const mpz_class&);

    // Static spellings on mpz_class:
    static mpz_class mpz_class::factorial (const mpz_class&);
    static mpz_class mpz_class::primorial (const mpz_class&);
    static mpz_class mpz_class::fibonacci (const mpz_class&);
    // plus generic overloads for any non-mpz operand.
}
```



```
}
```

10 User-Defined Literals

User-defined literals are defined under `gmpxx::literals` and `mpfrxx::literals`. Re-export into the parent namespace is *partial*: only the real-valued suffixes are re-exported. The imaginary-only complex suffixes `_mpfc_i` and `_mpc_i` must be brought in by an explicit `using namespace gmpxx::literals;` or `using namespace mpfrxx::literals;`.

Suffix	Type produced	Re-exported?	Notes
<code>_mpz</code>	<code>gmpxx::mpz_class</code>	yes	raw / auto-base; supports large hex literals
<code>_mpq</code>	<code>gmpxx::mpq_class</code>	yes	canonicalized; auto-base
<code>_mpf</code>	<code>gmpxx::mpf_class</code>	yes	floating and decimal string forms
<code>_mpfc_i</code>	<code>gmpxx::mpfc_class</code>	no	imaginary only; needs explicit <code>using namespace gmpxx::literals</code>
<code>_mpfr</code>	<code>mpfrxx::mpfr_class</code>	yes	floating and MPFR auto-base string forms
<code>_mpc_i</code>	<code>mpfrxx::mpc_class</code>	yes	imaginary only

“Re-exported” means the suffix is brought into the parent namespace (`gmpxx` or `mpfrxx`) automatically by including the relevant header.

```
using namespace gmpxx; // brings _mpz, _mpq, _mpf into scope
using namespace gmpxx::literals; // additionally brings _mpfc_i
using namespace mpfrxx; // brings _mpfr and _mpc_i into scope

auto z = 0xdeadbeefdeadbeefdeadbeef_mpf; // large hex via raw parser
auto q = "355/113"_mpq;
auto f = 3.14_mpf;
auto c = 1.0_mpf + 2.0_mpfc_i; // GMP-only complex
auto r = 3.14159265358979323846_mpfr;
auto z2 = 1.0_mpfr + (-3.25_mpc_i); // MPC complex
```

Real-only `_mpfc` and `_mpc` literals are intentionally absent; complex literals follow the standard library’s imaginary-only suffix design (`std::complex_literals`).

11 String and Stream I/O

11.1 Exact types (`mpz_class`, `mpq_class`)

```
namespace gmpxx {
    // string getters
    std::string mpz_class::get_str(int base = 10) const;
    std::string mpz_class::to_string() const;
    std::string mpq_class::get_str(int base = 10) const;
    std::string mpq_class::to_string() const;
```

```

// setters: set_str() is non-throwing and preserves value on failure;
// set() throws std::invalid_argument on failure.
bool mpz_class::set_str(const char* s, int base);
bool mpq_class::set_str(const char* s, int base); // rejects 0 denominator
void mpz_class::set(const char* s, int base = 10);
void mpq_class::set(const char* s, int base = 10);
}

```

`std::ostream` insertion supports decimal/hex/octal base selection, `showbase`, `uppercase`, `showpos`, `width`, `fill`, and `alignment`.

11.2 MPF

```

gmpxx::mpf_class f("1.0e-100", 1024);
f.set("3.14159");
f.set(std::string("3.14159"));
std::string s1 = f.get_str();
std::string s2 = f.get_str(/*exp out*/, /*base*/, /*digits*/);
bool ok = f.set_str("0xff.0p-4", /*base*/ 0); // value preserved on failure

```

`std::ostream` insertion uses `gmp_asprintf` and honors basic ostream formatting flags (precision, fixed/scientific, uppercase, showpoint, showpos, width, fill, alignment). The active C++ locale's decimal point is honored.

11.3 MPFR

`mpfr_class` string construction and string assignment throw `std::invalid_argument` on parse failure, matching MPF behavior. `set_str()` is non-throwing and preserves the destination value on failure. `std::ostream` insertion supports the standard ostream formatting flags.

11.4 MPFC complex

`mpfc_class` stream insertion materializes `mpfc` expressions through their components and is locale-aware via the underlying MPF I/O. Equality and inequality comparison (`operator==`, `operator!=`) are defined for `mpfc_class` pairs and for `mpfc_class` *vs.* `mpf_class`. No ordering operators (`<`, `<=`, etc.) are defined.

11.5 Stream extraction (`operator>>`)

Stream extraction is implemented for every wrapper type:

Type	Accepted input
<code>gmpxx::mpz_class</code>	integer literal in the stream's current base
<code>gmpxx::mpq_class</code>	num/den or integer (rejects zero den.)
<code>gmpxx::mpf_class</code>	floating-point literal
<code>gmpxx::mpfc_class</code>	(real,imag)
<code>mpfrxx::mpc_class</code>	(real,imag)

On parse failure, the destination value (and, for complex types, the per-component precision) is preserved and the stream's failbit is set. Raw-pointer overloads of `operator<<` and `operator>>` are also defined for `mpz_srcptr` / `mpz_ptr`, `mpq_srcptr` / `mpq_ptr`, and `mpf_srcptr` / `mpf_ptr`.

12 Math Functions

All math functions are eager: they accept object or expression operands and return a freshly constructed numeric object. They do *not* return expression nodes. Precision policy:

- Unary functions preserve the materialized operand precision.
- Binary and ternary functions use the maximum operand precision.
- Functions taking only scalar arguments come in two overloads: one with an explicit destination precision, one using the wrapper default precision (e.g., `sqrt_ui(value, prec)` versus `sqrt_ui(value)`).
- Pair-returning functions (`sin_cos`, `sinh_cosh`, `lgamma`) return both values at the materialized operand precision.

12.1 MPF (`gmpxx::mpf_class`)

Header: `<gmpxx_mkII.h>`.

Algebraic / power

`sqrt`, `abs`, `pow`.

Exponential / logarithm

`exp`, `exp2`, `exp10`, `expm1`; `log`, `log2`, `log10`, `log1p`.

Trigonometric

`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`.

Hyperbolic

`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`.

Special

`gamma`, `reciprocal_gamma`.

Constants

`const_pi`, `const_e`, `const_log10`, `log_two`, `inv_log_two`, `log_ten`, `pi_over_two`, `pi_over_four`, `two_pi`.

Implementation notes. All listed unary functions evaluate at the destination precision plus guard bits, then truncate the result back to the destination. Algorithms used:

- `const_pi` / `const_e` / `log_two`: hardcoded table lookup at 512, 1024, and 2048 bits when the requested precision matches; otherwise Gauss–Legendre (`pi`) or theta–AGM (`log_two`) at the requested precision.
- Trigonometric functions: argument reduction modulo $\pi/2$ at an internal constant precision, followed by Taylor series at working precision.
- `log`, `log1p`: hybrid path that switches between Taylor on $\log(1+x)$ for small $|x|$ and an `artanh`-series form for larger inputs, with cancellation-aware working-precision selection.
- `exp`, `expm1`: range-reduced Taylor series.
- `gamma`: Spouge’s algorithm with reflection for non-positive arguments. The number of Spouge terms is selected from the target precision; the function throws `std::domain_error` at non-positive integer poles.
- `sqrt`: Newton iteration via GMP’s `mpf_sqrt`.

For applications that require correctly-rounded (last-bit accurate) results, use `mpfrxx::mpfr_class` (Section 12.3), whose math wrappers delegate directly to MPFR.

Expression-input overloads. Every `mpf_class` math function above has a parallel template overload accepting any expression node whose `result_type` is `mpf_class`; the expression is materialized first and the function is then applied.

12.2 MPFC (`gmpxx::mpfc_class`)

Header: `<gmpxx_mkII.h>`.

Algebraic

`sqrt`; `abs`, `norm`, `arg`.

Exponential / logarithm

`exp`, `log`.

Trigonometric

`sin`, `cos`, `tan`, `asin`, `acos`, `atan`.

Hyperbolic

`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`.

Component / structure

`conj`, `real`, `imag`, `polar(radius, angle)`.

Power

Multiple `pow` overloads: `pow(mpf_class, std::uint64_t)`, `pow(mpf_class, std::int64_t)`, generic integral overloads, `pow(mpf_class, mpf_class)`, `pow(mpf_class, mpfc)`, `pow(mpf_class, mpfc)`, and a scalar/mpfc overload guarded by `is_supported_mpf_scalar_v`.

Each unary function additionally has an expression-input overload constrained by `is_expression_node_v` with `result_type == mpfc_class`, materializing the expression first.

Notes. `abs` returns `mpf_class` (i.e. $\sqrt{a^2 + b^2}$), `norm` returns `mpf_class` (i.e. $a^2 + b^2$), `arg` returns `mpf_class`, and `real/imag` return `mpf_class`. `conj` returns `mpfc_class`.

Removed. `mpfc_class` `gamma` and `reciprocal_gamma` were intentionally removed because the implementation was double-backed via `std::complex<double>` Lanczos and could not honor MPF precision. They will be reintroduced only with a genuine high-precision implementation.

12.3 MPFR (`mpfrxx::mpfr_class`)

Header: `<mpfrxx_mkII.h>`. All wrappers delegate to MPFR (verified against MPFR 4.x) at the materialized operand precision. The rounding mode used is `mpfr_class::default_rounding()`, i.e. the wrapper-owned default. Each function accepts both `mpfr_class` objects and expression nodes whose `result_type` is `mpfr_class`, `mpz_class`, or `mpq_class`; exact-typed expressions are first evaluated in the GMP family and then converted once via `mpfr_set_z` or `mpfr_set_q`.

Algebraic and ternary

`sqrt`, `sqr`, `sqrt_ui` (default- and explicit-precision overloads), `rec_sqrt`, `cbirt`, `rootn_ui`, `rootn_si`, `agm`, `neg`;

`fma(a, b, c)`, `fms(a, b, c)`;

`fmma(a, b, c, d)`, `fmms(a, b, c, d)`;

`hypot`, `min`, `max`, `dim`.

(`mpfr_root` is intentionally not wrapped because MPFR deprecates it; use `rootn_ui` or `rootn_si`.)

Power

`pow(lhs, rhs)`, `powr(lhs, rhs)`; `pow_si`, `pow_ui`, `pow_z`, `pow_sj` (`intmax_t` exponent), `pow_uj` (`uintmax_t` exponent); `ui_pow(base, expr)`, `ui_pow_ui(base, expr)`; `compound_si(expr, exponent)`.

Exponential and logarithm

`log`, `log2`, `log10`, `log1p`, `log2p1`, `log10p1`, `log_ui`; `exp`, `exp2`, `exp10`, `expm1`, `exp2m1`, `exp10m1`; `eint`, `li2`.

Trigonometric

`sin`, `cos`, `tan`; `asin`, `acos`, `atan`, `atan2(y, x)`, `atan2pi(y, x)`, `atan2u(y, x, unit)`; `sec`, `csc`, `cot`; `sinu(expr, unit)`, `cosu(expr, unit)`, `tanu(expr, unit)`, `asinu(expr, unit)`, `acosu(expr, unit)`, `atanu(expr, unit)`; `sinpi`, `cospi`, `tanpi`, `asinpi`, `acospi`, `atanpi`.

Hyperbolic

`sinh`, `cosh`, `tanh`; `asinh`, `acosh`, `atanh`; `sech`, `csch`, `coth`.

Constants (default- and explicit-precision overloads)

`const_pi`, `const_e`, `const_log2`, `const_log10`, `pi_over_two`, `const_euler`, `const_catalan`.

Sign and next-representable

`abs`, `signbit`, `sgn`, `setsign(expr, sign)`, `copysign(value, sign_source)`; `nextabove`, `nextbelow`, `nexttoward(expr, direction)`, `nextafter(expr, direction)`.

Predicates

`nan_p`, `inf_p`, `number_p`, `integer_p`, `zero_p`, `regular_p`.

Comparison helpers

`cmp(lhs, rhs)` (returns `int`, sign of `lhs - rhs`); `cmpabs(lhs, rhs)`, `cmpabs_ui(expr, ui)`, `reldiff`, `eq(lhs, rhs, bits)`; `greater_p`, `greaterequal_p`, `less_p`, `lessequal_p`, `lessgreater_p`, `equal_p`, `unordered_p`.

Pair-returning

`sin_cos(expr)` returns `std::pair<mpfr_class, mpfr_class>`; `sinh_cosh(expr)` returns `std::pair<mpfr_class, mpfr_class>`; `lgamma(expr)` returns `std::pair<mpfr_class, int>` (value plus sign); `remquo(lhs, rhs)` returns `std::pair<mpfr_class, long>` (remainder plus quotient bits).

Modular and remainder

`remainder(lhs, rhs)`, `fmod(lhs, rhs)`.

Special functions

`erf`, `erfc`, `gamma`, `reciprocal_gamma`, `lngamma`, `digamma`, `zeta`, `j0`, `j1`, `y0`, `y1`, `ai`; `zeta_ui`, `fac_ui` (each with default- and explicit-precision overloads); `jn(order, expr)`, `yn(order, expr)`, `gamma_inc(lhs, rhs)`, `beta(lhs, rhs)`.

Comparison operators

The full set of comparison operators `==`, `!=`, `<`, `<=`, `>`, `>=` is defined for `mpfr_class` against any combination of `mpfr_class` object, expression, or supported scalar (in either operand position). The implementation uses the named `*_p` predicates above; in particular `a < b` is `mpfr_less_p(a, b)`, which returns `false` if either operand is NaN. Equality on NaN is `false` (consistent with IEEE).

12.4 MPC (`mpfrxx::mpc_class`)

Header: `<mpcxx_mkII.h>`. All wrappers delegate to the MPC 1.3.1 C API at the materialized operand precision and at `mpc_class::default_rounding()`.

Unary, returning `mpc_class`

`sqrt`, `sqr`; `exp`, `log`, `log10`; `sin`, `cos`, `tan`; `sinh`, `cosh`, `tanh`; `asin`, `acos`, `atan`; `asinh`, `acosh`, `atanh`; `conj`, `proj`.

Unary, returning `mpfr_class`

`real`, `imag`, `abs`, `norm`, `arg`.

Binary and ternary

`pow(lhs, rhs)`, `agm(lhs, rhs)`; `fma(a, b, c)`.

Pair-returning

`sin_cos(z)` returns `std::pair<mpc_class, mpc_class>`.

Constants

```
mpc_class rootofunity(unsigned long order, unsigned long index);
mpc_class rootofunity(unsigned long order, unsigned long index,
                      mpfr_prec_t precision);
mpc_class rootofunity(unsigned long order, unsigned long index,
                      mpfr_prec_t real_precision,
                      mpfr_prec_t imag_precision);
```

Not provided. MPC 1.3.1 lacks `mpc_gamma` and `mpc_rgamma`; the wrapper does not provide complex gamma for `mpc_class`. Vector-reduction APIs (`mpc_sum`, `mpc_dot`) are not wrapped. Specialized `mpc_pow_*` fast paths (`mpc_pow_fr`, `mpc_pow_d`, `mpc_pow_si`, `mpc_pow_ui`, `mpc_pow_z`) are not separately exposed; the public `pow` materializes operands to `mpc_class` and calls `mpc_pow`. `mpc_mul_i`, `mpc_mul_2ui`, `mpc_div_2ui`, `mpc_mul_2si`, and `mpc_div_2si` are covered by general arithmetic and scalar expression routes rather than named wrappers.

13 Random Number API

There are two random-number wrapper classes, both named `gmp_randclass` but living in different namespaces with disjoint purposes.

13.1 GMP random class (gmpxx::gmp_randclass)

Header: <gmpxx_mkII.h>. RAII wrapper around gmp_randstate_t, non-copyable and non-movable (state ownership).

```
namespace gmpxx {
class gmp_randclass {
public:
    // Constructors
    gmp_randclass(); // default
    explicit gmp_randclass(decltype(gmp_randinit_default));
    explicit gmp_randclass(decltype(gmp_randinit_mt));
    gmp_randclass(decltype(gmp_randinit_lc_2exp_size), mp_bitcnt_t);
    gmp_randclass(decltype(gmp_randinit_lc_2exp), const mpz_class&,
                  unsigned long, mp_bitcnt_t);
    gmp_randclass(gmp_randalg_t, mp_bitcnt_t); // obsolete form

    // Seeding
    void seed(unsigned long s);
    void seed(const mpz_class& s);

    // Integer generation
    mpz_class get_z_bits(mp_bitcnt_t n);
    mpz_class get_z_bits(const mpz_class& n);
    mpz_class get_z_range(const mpz_class& upper);

    // Float generation (returns an expression node so the caller's
    // destination precision drives generation)
    random_mpf_expr get_f();
    random_mpf_expr get_f(mp_bitcnt_t prec);
    random_mpf_expr get_f(const mpf_class& prototype);
};
}
```

Negative bit counts, non-positive range limits, and over-large LC table sizes are rejected with errors.

13.2 MPFR random class (mpfrxx::gmp_randclass)

Header: <mpfrxx_mkII.h>. As MPFR documents, MPFR random generation runs on a GMP gmp_randstate_t.

```
namespace mpfrxx {
class gmp_randclass {
public:
    // Constructors mirror gmpxx::gmp_randclass (default, MT, LC variants,
    // obsolete gmp_randalg_t form).

    void seed(unsigned long s);
    void seed(const gmpxx::mpz_class& s);

    mpz_class get_z_bits(mp_bitcnt_t n);
    mpz_class get_z_bits(const mpz_class& n);
    mpz_class get_z_range(const mpz_class& upper);

    // [0, 1) via mpfr_urandomb (MPF-style)
    random_mpfr_expr get_fr();
    random_mpfr_expr get_fr(mpfr_prec_t prec);
};
}
```

```

random_mpfr_expr get_fr(const mpfr_class& prototype);

// Distributions
random_mpfr_expr get_fr_uniform(); // mpfr_urandom
random_mpfr_expr get_fr_uniform(mpfr_prec_t);
random_mpfr_expr get_fr_normal(); // mpfr_nrandom
random_mpfr_expr get_fr_normal(mpfr_prec_t);
random_mpfr_expr get_fr_exponential(); // mpfr_erandom
random_mpfr_expr get_fr_exponential(mpfr_prec_t);

// MPFR C-API name aliases
random_mpfr_expr get_fr_urandomb(); // == get_fr
random_mpfr_expr get_fr_urandom(); // == get_fr_uniform
random_mpfr_expr get_fr_nrandom(); // == get_fr_normal
random_mpfr_expr get_fr_erandom(); // == get_fr_exponential
};
}

```

The expression nodes `random_mpfr_expr` and `random_mpfr_expr` integrate with the standard expression-template materialization so that construction uses the requested or wrapper-default precision and assignment preserves the destination precision. Non-zero MPFR ternary return values from `mpfr_urandom`, `mpfr_nrandom`, and `mpfr_erandom` are rounding information, not failures.

14 Examples

The `examples/` directory contains the following programs. Each is registered as a CTest target.

Program	Demonstrates	Backend
example01	Explicit-precision MPF construction, expression-template addition, stream output.	MPF
example02_mpf	Default-precision MPF basics.	MPF
example02_mpfr	Default-precision MPFR basics.	MPFR
example02_mpfr_mpc	Combined MPFR/MPC use through the aggregator header.	MPFR+MPC
example03_mpf	Newton/Heron iteration for $\sqrt{2}$.	MPF
example03_mpfr	Newton iteration parallel to the MPF version.	MPFR
example03_mpfc_math	Eager MPFC math helpers.	MPFC
example04_*	Port of upstream example04 to MPF and MPFR.	MPF / MPFR
example05_*	Port of upstream example05.	MPF / MPFR
example06_mpfc	Complex example via MPFC.	MPFC
example06_mpc	Complex example via MPC pair constructor.	MPC
example07_mpfc	ASCII Mandelbrot using MPFC.	MPFC
example07_mpc	ASCII Mandelbrot using MPC.	MPC
example08_*	Wilkinson polynomial conditioning.	MPFC / MPC
example09_mpfc	Aberth–Ehrlich root-finding (default precision).	MPFC
example09_mpc	Aberth–Ehrlich root-finding (default precision).	MPC
example10_*	Comment-only license-aligned port.	MPF / MPFR
example11_*	Upstream example11 port.	MPF / MPFR
example12_*	Upstream example12 port.	MPF / MPFR
example13_*	Upstream example13 port.	MPF / MPFR
example14_*	Upstream example14 port.	MPF / MPFR

Program	Demonstrates	Backend
example15_mpf	BBP-style hexadecimal-digit extraction.	MPF
example15_mpfr	BBP-style hexadecimal-digit extraction.	MPFR

A minimal program using the aggregator header looks like:

```
#include <gmpfrxx_mkII.h>
#include <iostream>

int main() {
    using namespace gmpxx::literals;
    using namespace mpfrxx::literals;

    auto a = 3.14_mpf;
    auto b = 2.71_mpfr;
    mpfrxx::mpfr_class c = a * b + 1;
    std::cout << c << '\n';

    mpfrxx::mpc_class z(a, b);
    z = mpfrxx::sin(z);
    std::cout << z.real_to_double() << ' '
              << z.imag_to_double() << '\n';
    return 0;
}
```

15 Limitations and Notes

The following limitations reflect the current state of the library and are recorded explicitly so users can avoid surprises.

- **MPF transcendentals are not correctly rounded.** The `gmpxx::mpf_class` math wrappers compute at the destination precision plus internal guard bits using arbitrary-precision algorithms (see Section 12.1), but the results are not guaranteed last-bit correct. For correctly-rounded transcendentals, use `mpfrxx::mpfr_class`.
- **MPFC complex gamma removed.** The previous implementation was double-backed via Lanczos and could not honor MPF precision. It will be reintroduced only with a genuine high-precision implementation.
- **MPC gamma not provided.** MPC 1.3.1 has no `mpc_gamma` or `mpc_rgamma`.
- **Exact division by zero** follows the underlying GMP behavior; no wrapper-specific diagnostic is raised.
- **MPF effective precision** as reported by `mpf_get_prec()` may exceed the requested value because GMP rounds up to its internal limb allocation.
- **Compile-time-rejected expression-template scalar leaves:** `bool`, `long double`, `__int128`, `unsigned __int128`. Note that `mpz_class` accepts `__int128_t` and `__uint128_t` *directly* as constructor arguments; the rejection applies only when these types appear inside an arithmetic expression (e.g. `mpf + __int128_t{...}`).
- **Forbidden mixed families:** `mpf` with `mpfr`; `mpf` with `mpc`; `mpfc` with `mpfr`; `mpfc` with `mpc`.

- **Standard-library traits are partial.** `std::numeric_limits` is specialized only for `mpz_class`, `mpq_class`, and `mpf_class` (with the minimal flags listed in the standard-library interoperability section). For `mpfc_class`, `mpfr_class`, and `mpc_class`, `is_specialized` is false. `std::common_type` is specialized for the GMP-side classes paired with builtin scalars and for cross-class promotions; on the MPFR/MPC side, only the three `mpfr_class/mp{z,q,fr}_class` pairings are specialized — scalar pairings are not.
- **No MPC vector reductions.** `mpc_sum` and `mpc_dot` are unwrapped; ditto `mpfr_sum` and `mpfr_dot`. These require a separate public container/API policy that has not been settled.
- **Random statistical quality is not certified.** The bundled smoke checks catch gross wiring errors only.

A Public Symbol Index

Namespace `gmpxx`

Types: `mpz_class`, `mpq_class`, `mpf_class`, `mpfc_class`, `gmp_randclass`, `random_mpf_expr`.
 Defaults: `default_mpf_precision_bits`, `set_default_mpf_precision_bits`,
`reload_default_mpf_precision_bits_from_environment`.
 Version info: `git_commit_hash`, `print_git_commit_hash`.
 Math (`mpf_class`): see Section 12.1.
 Math (`mpfc_class`): see Section on MPFC math.
 Number-theoretic (`mpz_class`): `abs`, `sqrt`, `gcd`, `lcm`, `factorial`, `primorial`, `fibonacci`.
 Literals (in `gmpxx::literals`; `_mpz`, `_mpq`, `_mpf` also visible directly in namespace `gmpxx`, `_mpfc_i` only via using namespace `gmpxx::literals`): `_mpz`, `_mpq`, `_mpf`, `_mpfc_i`.

Namespace `mpfrxx`

Types: `mpfr_class`, `mpc_class`, `mpz_class` (alias of `gmpxx::mpz_class`), `mpq_class` (alias of `gmpxx::mpq_class`), `gmp_randclass`, `random_mpfr_expr`.
 MPFR defaults: `default_options`, `default_precision_bits`,
`default_prec`, `set_default_precision_bits`, `default_rounding_mode`,
`set_default_rounding_mode`, `default_emin`, `default_emax`, `set_default_exponent_range`,
`reload_mpfr_defaults_from_environment`.
 MPC defaults: `default_mpc_options`, `default_mpc_precision_bits`,
`default_mpc_real_precision_bits`, `default_mpc_imag_precision_bits`,
`set_default_mpc_precision_bits`, `default_mpc_rounding_mode`,
`default_mpc_real_rounding_mode`, `default_mpc_imag_rounding_mode`,
`set_default_mpc_rounding_mode`, `reload_mpc_defaults_from_environment`.
 Version info: `git_commit_hash`, `print_git_commit_hash`.
 Math (`mpfr_class`): see Section 12.3.
 Math (`mpc_class`): see Section 12.4.
 Literals (in `mpfrxx::literals`; `_mpfr` and `_mpc_i` also visible directly in namespace `mpfrxx`):
`_mpfr`, `_mpc_i`.

Macros

`GMPFRXX_MKII_ASSUME_FIXED_PRECISION_FASTPATH`—enable the fixed-precision evaluation fast path.
`GMPFRXX_MKII_GIT_COMMIT_HASH`—string literal of the build’s Git commit hash, also exposed as `gmpfrxx_mkII::detail::git_commit_hash`.

Environment variables

MPFXX_DEFAULT_PREC_BITS, MPFRXX_DEFAULT_PRECISION_BITS, MPFRXX_EMIN,
MPFRXX_EMAX, MPFRXX_ROUNDING_MODE, MPFRXX_MPC_DEFAULT_PRECISION_BITS,
MPFRXX_MPC_REAL_PRECISION_BITS, MPFRXX_MPC_IMAG_PRECISION_BITS,
MPFRXX_MPC_ROUNDING_MODE, MPFRXX_MPC_REAL_ROUNDING_MODE,
MPFRXX_MPC_IMAG_ROUNDING_MODE.